FAME-QP

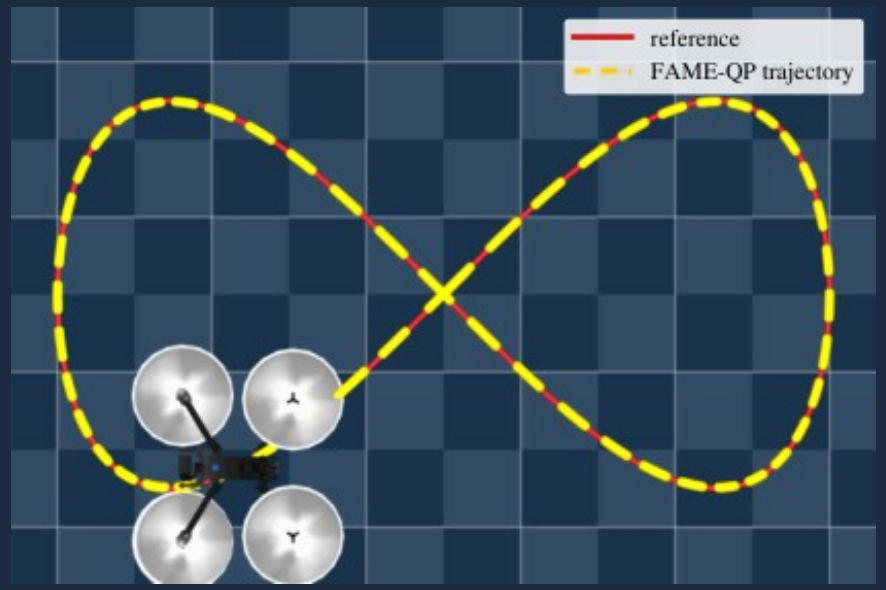
面向实时模型预测控制的 GPU 加速二次规划求解器

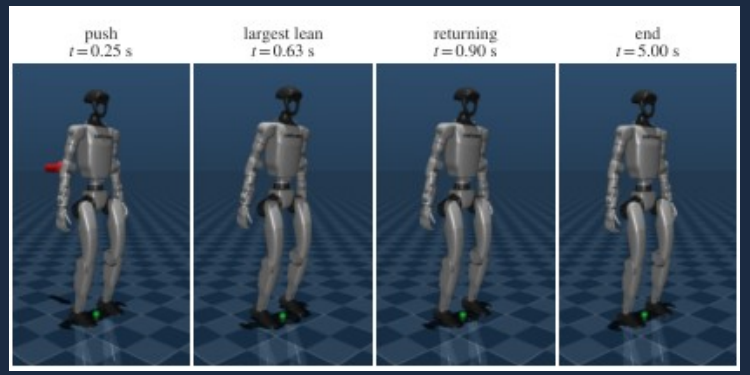
A GPU-Accelerated Quadratic-Programming Solver for Real-Time Model Predictive Control

Santosh Mohan Rajkumar · Debdipta Goswami · Stephanie Stockar
The Ohio State University

arXiv 2611.13766 （2026-11-10） · 开源： github.com/santoshrajkumar/fame-qp

论文解读





图源：论文 Fig. 2 / Fig. 3 （MuJoCo 仿真）

一句话总结

算法不变，把 ADMM 在 GPU 上的执行开销逐项压下去

FAME-QP 以 ADMM + 预求逆罚参数库为骨架，通过 **约束结构感知算子、融合单 / 双核迭代 + CUDA Graph 回放、设备标定与实测驱动的自动配置、罚参数探测与惰性矩阵更新**，在随机 QP 、随机 LMPC 、人形与四旋翼 MPC 上比现有 GPU 求解器显著降低延迟，同时保持精度和控制效果。

1.5–4.3×

随机稠密 QP

相对 ReLU-QP （n=220–1400，同精度）

3.2–7.3×

随机系统 LMPC

相对 ReLU-QP，每步 0.66–4.23 ms

N 10→20

Unitree G1 推扰恢复

2 ms 控制周期内可用预测时域翻倍

4.7×

四旋翼 Koopman MPC

相对 CuPIQP， P99 17.6 ms < 20 ms

MPC 每个周期都要解一次 QP：延迟决定了频率和时域

内点法 IP

HPIPM · PIQP · Clarabel

+ 迭代少
− 每步解大线性系统；难热启动

有效集法 AS

qpOASES · DAQP

+ 热启动高效
− 最坏复杂度随约束数组合增长

算子分裂 ADMM

OSQP · ProxQP

+ 易热启动、易并行
− 收敛较慢，适合中等精度

↑ FAME-QP 所属路线

为什么要 GPU 原生

- 世界模型、潜空间线性动力学、 Koopman 模型带来大规模 MPC
- 机器人仿真、推理、学习都已在 GPU 上运行
- CPU 求解器 ⇒ 频繁 CPU↔GPU 数据传输，增加延迟

最接近的工作 ReLU-QP 的不足 → FAME-QP 的对策

- 稠密 $(n+2m)^2$ 算子过大 → **约束结构感知的更小算子**
- 每次迭代多次 kernel 调度 → **融合 kernel + CUDA Graph 回放**
- 离线预计算，无法在线改矩阵 → **惰性算子更新**

每次 ADMM 迭代 = 2 个矩阵 - 向量乘 + 一串逐元素运算

凝聚（condensed） LMPC

用动力学 $X = \mathcal{A}\hat{x} + \mathcal{B}U$ 消去预测状态，只保留输入：

$z = U$， $n = N \cdot n_u$

$P = \mathcal{B}^T\mathcal{Q}\mathcal{B} + \mathcal{R}$， $q = \mathcal{B}^T\mathcal{Q}\mathcal{A}\hat{x}$

$G = [\mathcal{B}; I]$， $m = N(n_x+n_u)$

→ 稠密矩阵，适合 GPU 稠密线性代数

预处理

- 改进 Ruiz 均衡缩放（10 轮）
- 罚矩阵 $\boldsymbol{\rho}$ = ρ · diag(b)，等式行 $b = 10^3$
- 开环不稳定系统建议 LQR 预稳定

① $\tilde{z} = M \cdot r$
$M = (P+\sigma I+G^s\boldsymbol{\rho}G)^{-1}$ 预先求逆

② $\tilde{s} = G \cdot \tilde{z}$
矩阵 - 向量乘

③ $z \leftarrow \alpha\tilde{z} + (1-\alpha)z$
松弛（$\alpha = 1.6$）

④ $s = \Pi_C(\zeta)$
$\zeta = \alpha\tilde{s}+(1-\alpha)s+\boldsymbol{\rho}^{-1}\lambda$，逐元素裁剪到 $[l, h]$

⑤ $\lambda = \rho(\zeta - s)$
对偶更新；构造下一轮 r

耦合
矩阵 - 向量乘
⇒ 带宽受限

逐元素
可与矩阵乘融合在同一 kernel 中

贡献 ①：约束结构感知的算子

只把真正稠密的约束行放进算子：存储最多减 4×

Box

$G = D = \mathrm{diag}(d)$

来源：只有输入约束

存 M 和 d： $n^2 + n$ 个元素

（按稠密处理需 $3n^2–4n^2$ ）

Split

$G = [D ; C]$

来源：输入约束 + 状态约束

D 逐元素吸收进 r，

只有 C （m_c 行）按稠密处理

General

整行稠密， $C = G$

来源： LQR 预稳定后输入约束耦合

所有行按稠密处理，

$m_c = m$

两种算子形式的取舍（ Split / General ）

堆叠形式 $W = [M, MC^T ; CM, CMC^T]$： 一次矩阵乘同时得到 $\check{z}$、 $\check{s} \to 1$ 个 kernel

分解形式 $F = [M, MC^T]$ 加 C： 先算 $\check{z}$，再算 $C\check{z} \to 2$ 个 kernel

$(n+m_c)^2 - n(n+2m_c) = \mathbf{m_c^2}$ ← 分解形式每次迭代少读的元素数

消融（ $G = [I ; C]$ ）

存储 ↓ **1.67–4.00×**

迭代时间 ↓ **1.07–3.03×**

n=512, m_c=128： 10.1 → 3.12 MiB （ ReLU-QP 为 24.5 MiB ）

贡献 ②：融合执行 + CUDA Graph + 自动选择

一次迭代融合成 1–2 个 kernel，每 10 次迭代只和 CPU 交互一次

单核模式（堆叠算子 W）

W · r → 松弛 / 投影 / 对偶 → 构造下一 r

双核模式（分解算子 F, C）

Kernel 1
z̃ = F · r，更新 z 和对角约束

→

Kernel 2
s̃_c = C · z̃，投影 + 对偶更新

CUDA Graph 回放

- 录制 j_chk = 10 次迭代 + 终止检测（Pz、Gz、GᵀΛ、两个归约 kernel）
- host 每次回放只读 5 个标量：收敛 / 停滞 / 切换 ρ

自动模式选择：模型剪枝，实测决定

$$t_{iter} \approx n_{ker} \cdot \tau + t_{stream}(b_{op})$$

τ：每个 graph 节点开销；t_stream：读算子耗时（0.25–64 MB 标定，按设备缓存）

1. 排除显存放不下的形式（罚参数库 > 60% 可用显存）
2. 剪掉预测耗时 > 最优 1.5× 的形式；常带宽近似下 $m_c < \sqrt{(\tau \cdot BW/\beta)}$ 时堆叠更快
3. 在真实维度上用占位算子实测，同时调 tile 配置
4. 只有双核比单核快 10% 以上才选双核

贡献 ③：罚参数库、探测与在线更新

罚参数探测选好起点，惰性更新应对时变矩阵

罚参数库

- 预先求逆 5 个 ρ，间隔 10×，初始中心 $\rho_0 = 0.1$
- 残差平衡：$\rho_new = \rho \cdot \sqrt{(r_prim \cdot \eta_d / (r_dual \cdot \eta_p))}$
- 相差 >5× 才切换到最近库项，无需重新求逆
- λ 在切换时保留（最优值与 ρ 无关）

初始罚参数探测

- setup 时每个库项从零试跑 ⩽ 400 次迭代
- 比较达到最紧容差（1/100、1/10、1 倍）的耗时
- 最优在端点 → 重新居中再探测（最多 2 次）
- Split 结构再探测 $\gamma \in \{10^{-3} \ldots 10\}$，平衡稠密行权重

在线更新与精度

- q、l、h 变化：沿用已存缩放因子，重建 r
- P、G 变化：惰性模式只重算当前库项，其余标为过期
- 热启动：可将上一解沿时域平移一步
- 精度自动选：按 κ(K) 选 float32 / float64

代价：探测只在 setup 阶段做，求解时不再探测；消融中 setup 成本约需 36–122 次求解收回。罚参数网格在单次求解中固定。

实验 1： 稠密随机 QP 冷启动求解

同精度下快于所有 GPU 基线； n ⩾ 220 后也超过所有 CPU 求解器

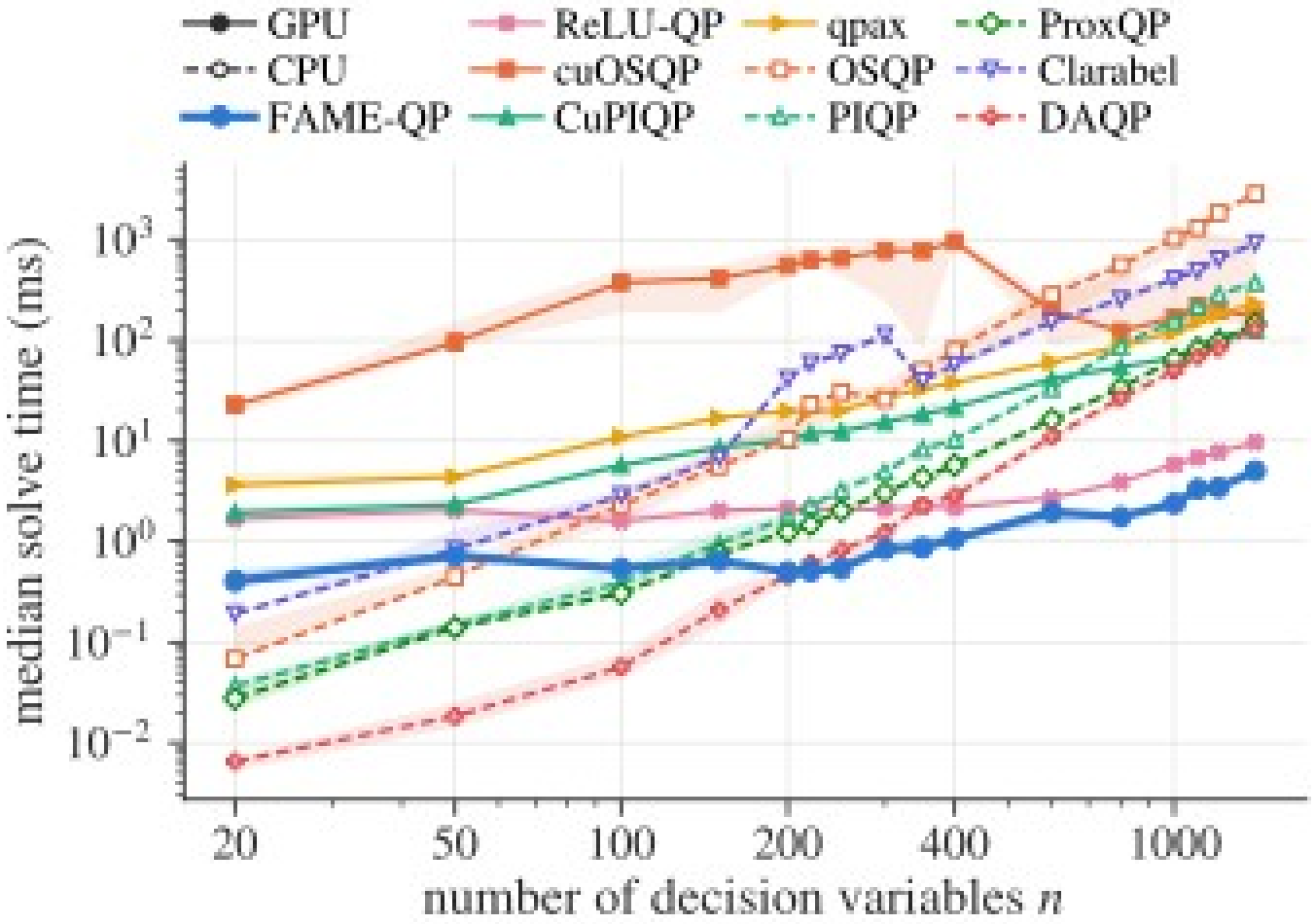


Fig. 1： 同精度冷启动求解时间（实线 GPU ，虚线 CPU ； cuOSQP 为 float32 ，其余 float64 ）

1.5–4.3× vs ReLU-QP

最强 GPU 基线， n = 220–1400

1.2–25.9× vs DAQP

最强 CPU 基线， n = 220–1400

n = 1400 时的平均求解时间

FAME-QP float32 **1.95 ms**

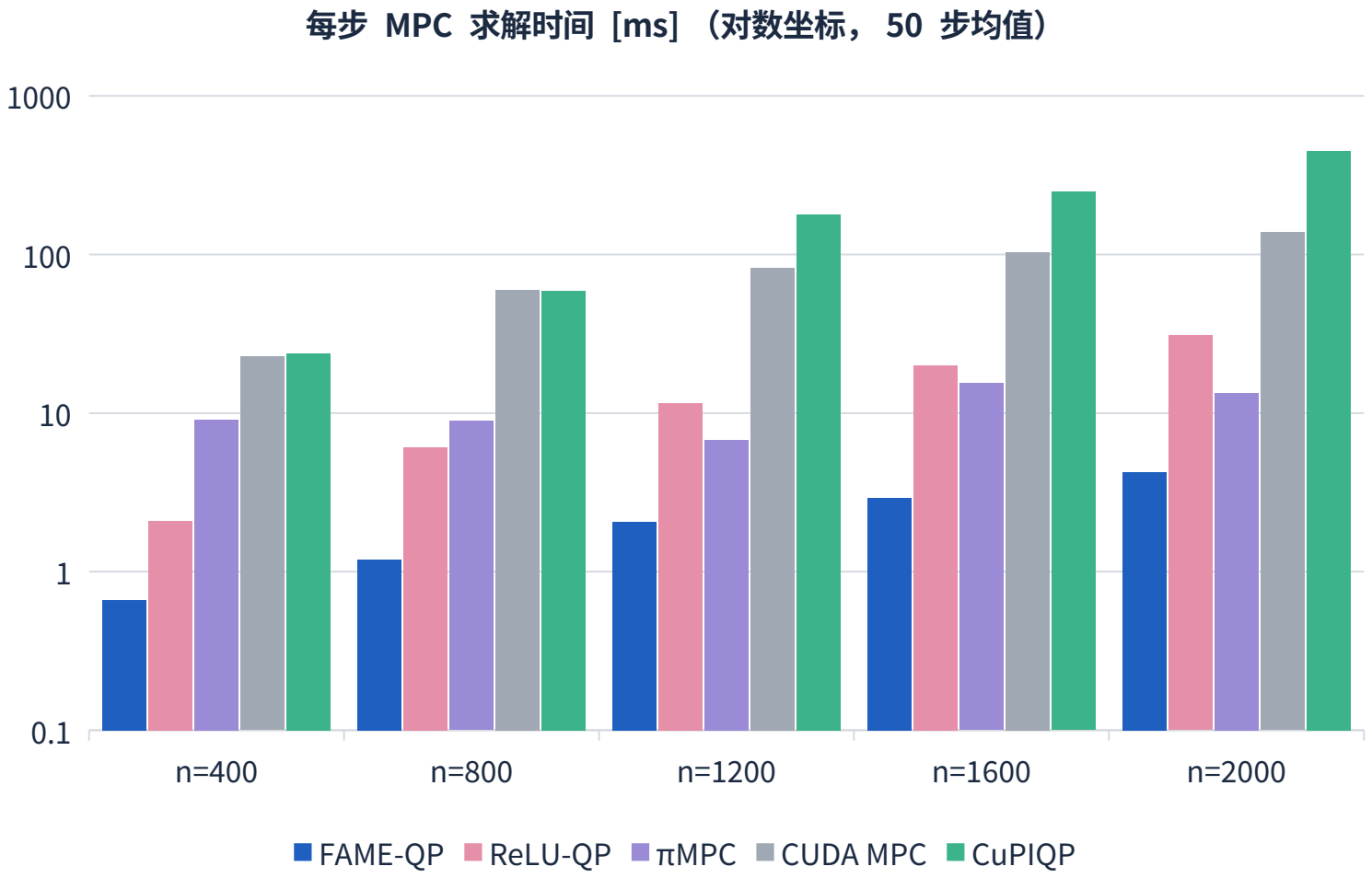
FAME-QP float64 **5.01 ms**

cuOSQP float32 **326 ms**

注意： n 较小时 DAQP 等 CPU 求解器更快， FAME-QP 有约 0.4–0.6 ms 的固定开销

实验 2： 随机不稳定系统上的 GPU LMPC （ N = 40 ）

闭环 LMPC： 每一档规模都最快，且时间波动最小



3.2–7.3×
vs ReLU-QP （最接近的 QP 基线）

3.2–13.8×
vs πMPC （最快的 MPC 基线）

精度对照（停止准则不等价）

- 闭环代价：所有方法相差 ⩽ 0.06%
- 最大约束违反： FAME-QP ⩽ 2.0×10^{-3}； ReLU-QP $1.4–4.3\times10^{-2}$； πMPC ≈ 10^{-4}； CuPIQP 、 CUDA MPC < 10^{-6}
- πMPC 标准差大于均值（如 9.06 ± 12.39 ms ）

实验 3： 人形机器人推扰恢复（MuJoCo）

Unitree G1： 2 ms 控制周期内，可用预测时域从 10 翻倍到 20

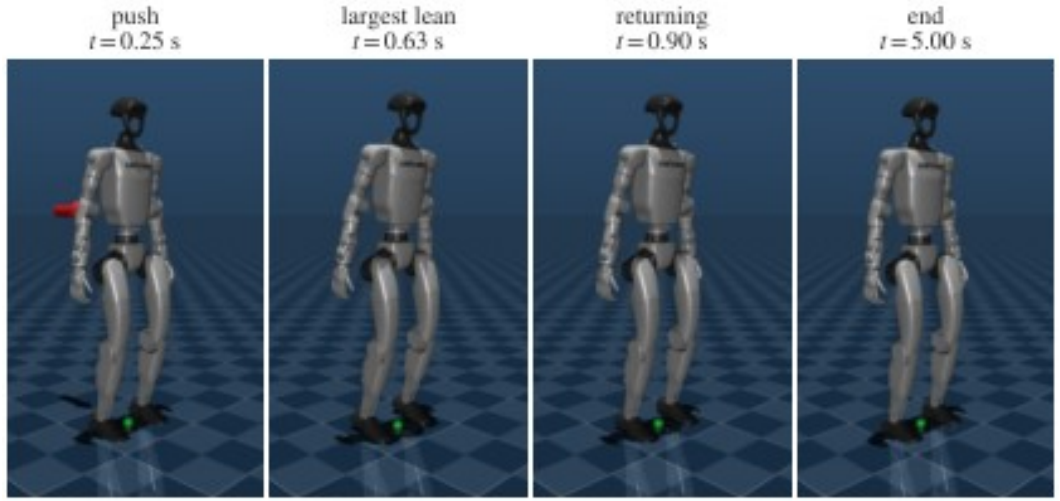
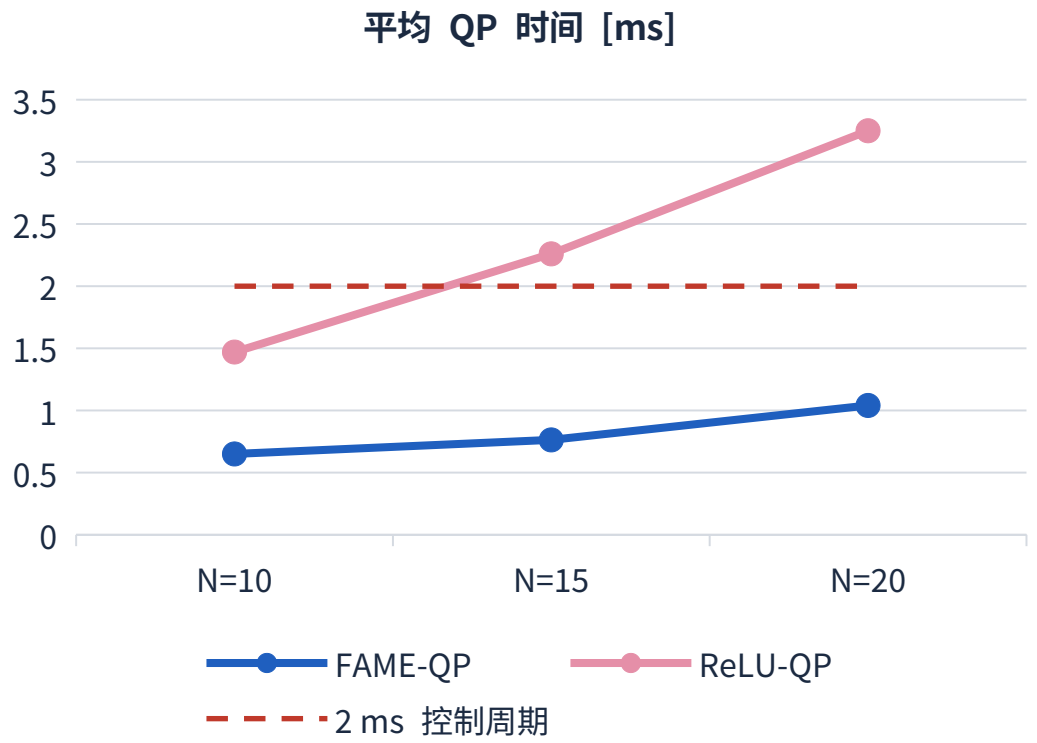


Fig. 2： 推扰 → 最大倾斜 → 回正 → 最终站立

29 关节 · 70 维状态 · 80 N 推力 0.1 s · 恢复 5 s · n = m = 29N

N (n)	FAME-QP [ms]	ReLU-QP [ms]	CoM RMS [mm]
10 (290)	**0.650 ± 0.091**	1.47 ± 0.05	18.7 / 18.7
15 (435)	**0.763 ± 0.116**	2.26 ± 0.12	19.1 / 19.2
20 (580)	**1.04 ± 0.20**	3.25 ± 0.13	20.5 / 19.5



QP 时间 ↓ 2.3–3.1×； ReLU-QP 在 N ⩾ 15 时超过 2 ms，平衡恢复效果两者相当

实验 4： Skydio X2 四旋翼 Koopman-LMPC （MuJoCo）

QP 矩阵每步都变： 4.7× 加速， P99 延迟仍低于控制周期

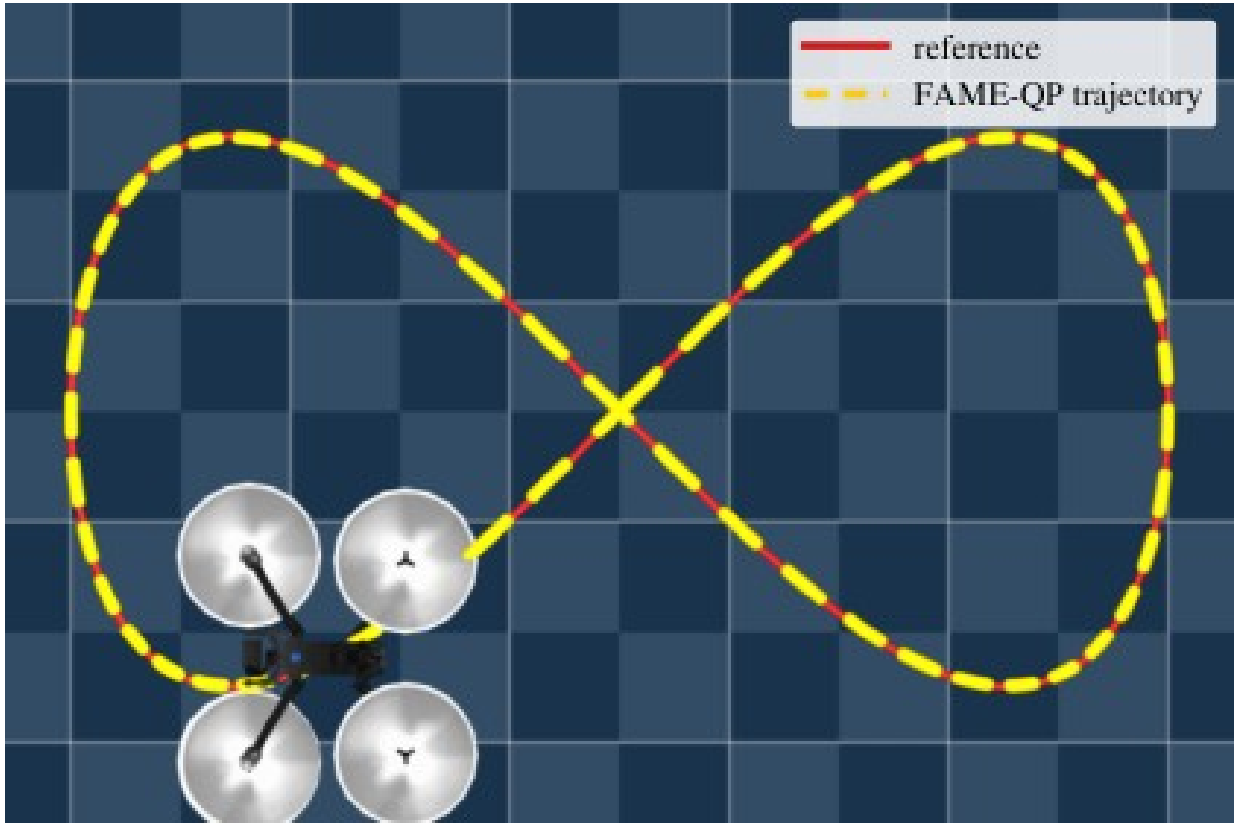
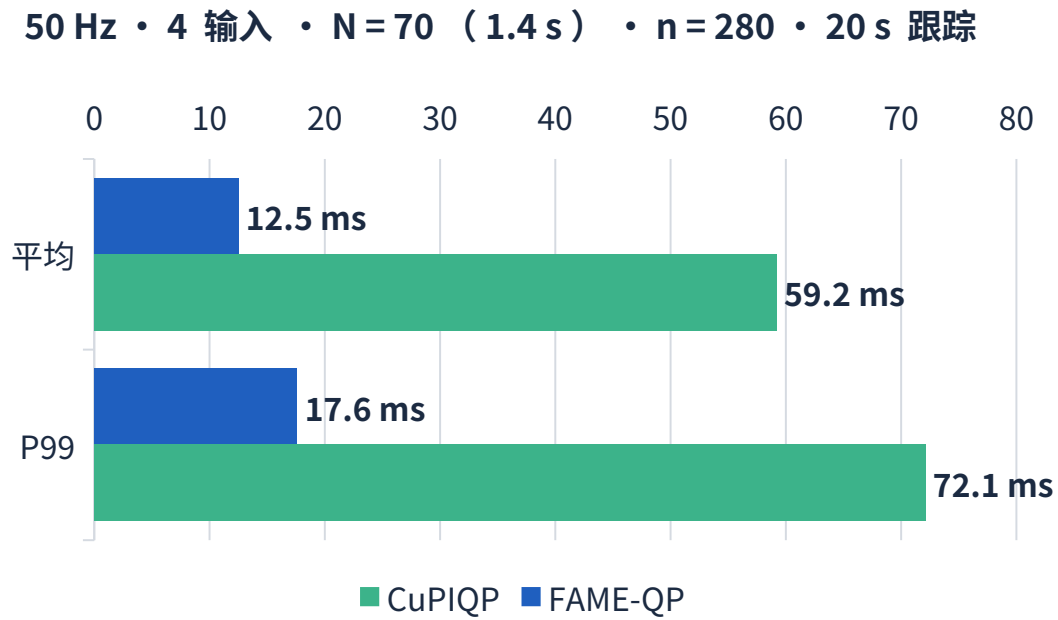


Fig. 3： 双纽线参考（红）与 FAME-QP 实际轨迹（黄虚线）

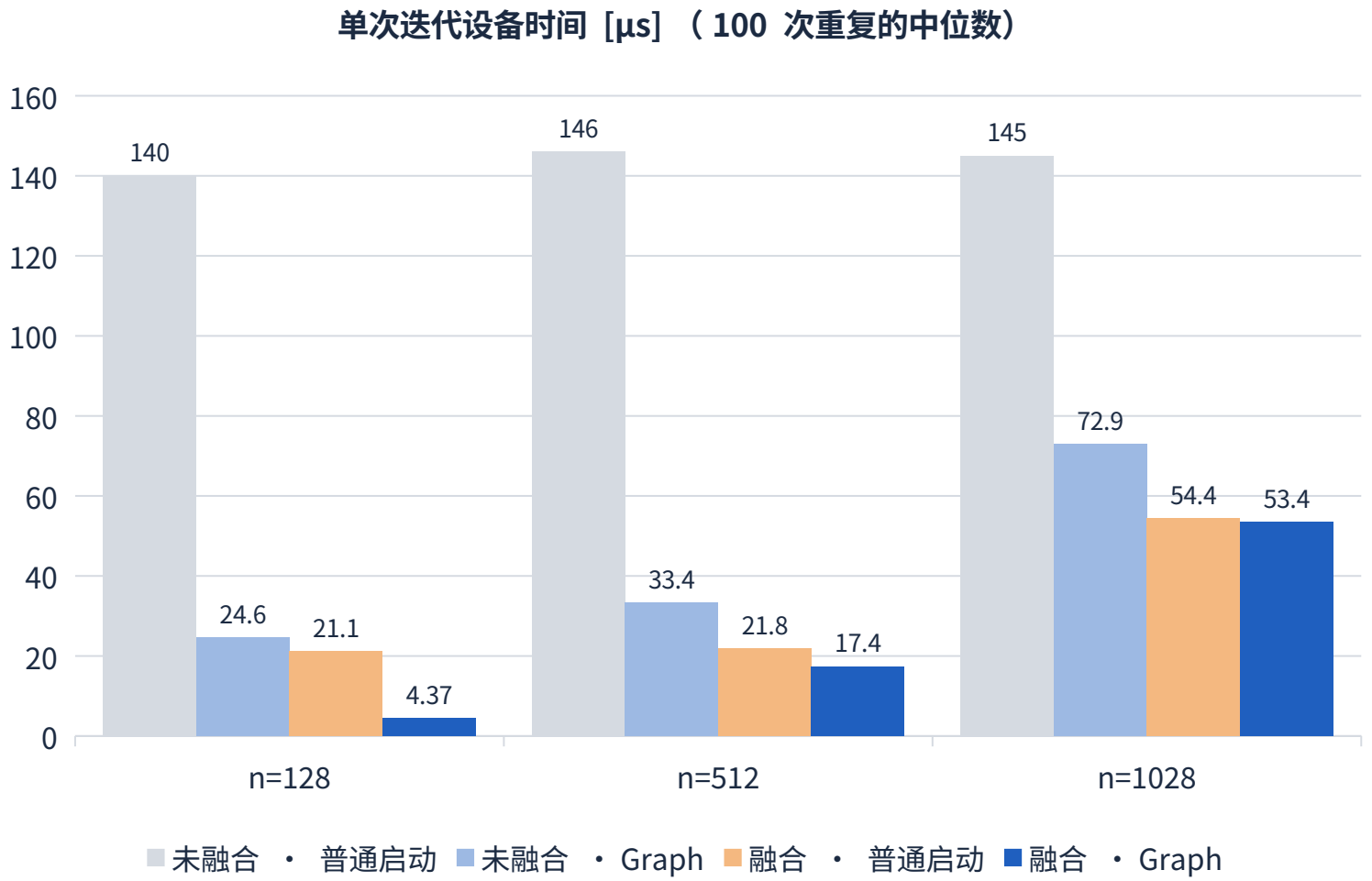


4.7×
平均 QP 时间加速

17.6 ms
P99 < 20 ms 控制周期

2.7 mm
位置 RMSE （CuPIQP 3.4 mm）

融合与 Graph 回放互补：小问题上单次迭代 140 µs → 4.37 µs



32×

n=128： 140 → 4.37 µs； 14 个 kernel 合并为 1 个

怎么读这张图

- Graph 回放消掉 host 端启动开销，融合减少 kernel 数和显存往返，二者叠加
- 问题越大，矩阵 - 向量乘（带宽）越占主导，收益越小： n=1028 时 72.9 → 53.4 µs
- 四种实现的迭代结果一致（误差 ⩽ 3×10^{-10}）

罚参数探测减少迭代次数，自动选择和惰性更新守住性能

初始罚参数探测（稠密随机 QP， $\varepsilon = 10^{-5}$）

n	迭代：默认	迭代：探测	时间：默认	时间：探测
250	74	46	1.10 ms	0.79 ms
600	98	48	3.78 ms	1.88 ms
1000	128	40	10.6 ms	3.21 ms

关键发现

- 融合 × 探测收益相乘： **2.8–4.2×** （融合 2.00→1.26×，探测 1.40→3.35×）
- 达到 10^{-3} 精度时， FAME-QP 需 30–38 次迭代， ReLU-QP 需 30–32 次

⇒ 相对 ReLU-QP 的加速**主要来自单次迭代更便宜**，而不是迭代次数更少

自动算子选择

36 个测试点中选中更快形式 30 次；选错时最多慢 9.5%（在 10% 保留阈值内）。固定只用堆叠或只用分解，最多慢 2.2× / 1.5×。单靠模型估计时交叉点偏差约 2×。

Split 结构的 γ 探测

四旋翼 KQ-LMPC 回放（N=50）：探测 ρ 降低中位与尾部步时；再探测 γ 时中位基本不变，尾部进一步缩短，求解中的罚参数切换几乎消失。

惰性矩阵更新

全量刷新时，重算 5 个库项成为每步主要开销；惰性刷新使更新时间和中位步时约减半，迭代次数、解和跟踪误差都不变。

评价：扎实的 GPU 工程设计，适用范围为中等规模的稠密 MPC

优点

✓ 只改执行、不改算法：执行开销被逐项拆解，每项设计都有消融支撑

✓ 模型剪枝 + 实测决定：论文也说明单靠模型会把交叉点估错约 2×

✓ 支持在线矩阵更新，适合 Koopman / 线性化的时变 MPC

✓ 同精度比较，并报告约束违反、 P99 和 setup 回收成本；代码开源

局限与疑问

! 仅面向稠密 QP （实验均用凝聚形式）：存储随 n^2 增长，罚参数库要存 5 份（作者把稀疏形式列为未来工作）

! $n < 200$ 时比 DAQP 等 CPU 求解器慢， GPU 固定开销约 0.4 ms

! ADMM 精度有上限：约束违反约 10^{-3}； float32 在 10^{-6} 附近会停滞

! setup 探测需要 36–122 次求解才能收回成本

! 只测了一款笔记本 GPU （ RTX 3080 Laptop ）；CUDA MPC 为作者复现； G1 实验只报告平均时间

要点回顾

什么问题	线性 MPC 凝聚后的稠密 QP，要求低延迟，并在 GPU 原生流水线中求解
怎么做	ADMM + 预求逆罚参数库；约束结构感知算子；融合单 / 双核迭代 + CUDA Graph；标定与实测驱动的自动配置；罚参数探测；惰性更新
效果如何	比最强 GPU 基线 ReLU-QP 快 1.5–7.3×（随机 QP / LMPC）；G1 在 2 ms 周期内时域从 10 增到 20；四旋翼在矩阵时变时 P99 仍低于控制周期
加速来源	主要是单次迭代更便宜，而非迭代次数更少；最适合同一形状反复求解、精度要求中等、对延迟敏感的场景

arXiv 2611.13766 • 代码： github.com/santoshrajkumar/fame-qp